Practical NO.08
Name - Sakshi Vijaykumar Muthiyan
Roll No. -
Title - Data Cleaning and Preparation
Problem Statement: Analyzing Customer Churn in a Telecommunications Company
Dataset: "Telecom_Customer_Churn.csv"
Description: The dataset contains information about customers of a telecommunications company and whether they have churned (i.e., discontinued their services). The dataset includes various attributes of the customers, such as their demographics, usage patterns, and account information. The goal is to perform data cleaning and preparation to gain insights into the factors that contribute to customer churn.

In [2]:
```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
# local path of telecom data on desktop
path = r"C:\Users\muthi\OneDrive\Desktop\PRACTICAL EXAM\DMV\WA_Fn-UseC_-T
print("Loading telecom dataset...")
try:
    df = pd.read_csv(path)
    # automatically clean names to fix spelling or capitalization differe
    df.columns = df.columns.str.strip().str.replace(' ', '_')
    rename_map = {
        'gender': 'Gender', 'tenure': 'Tenure_Months', 'customerid': 'cus
        'monthlycharges': 'Monthly_Charges', 'MonthlyCharges': 'Monthly_C
        'totalcharges': 'Total_Charges', 'TotalCharges': 'Total_Charges',
        'churn': 'Churn_Label', 'Churn': 'Churn_Label'
    }
    df = df.rename(columns=rename_map)
except FileNotFoundError:
    # fallback
    np.random.seed(42)
    mock_data = {
        'customerID': [f'ID-{i}' for i in range(1, 101)],
        'Gender': ['Male', 'Female', 'Male', 'Female', 'Female'] * 20,
        'Tenure_Months': np.random.randint(1, 72, 100).astype(float),
        'Monthly_Charges': np.random.uniform(20.0, 120.0, 100),
        'Total_Charges': np.random.uniform(100.0, 5000.0, 100),
        'Churn_Label': ['Yes', 'No', 'No', 'Yes'] * 25
    }
    df = pd.DataFrame(mock_data)
    df.loc[2, 'Monthly_Charges'] = np.nan # add nan on purpose for task 3
    df.loc[5, 'Tenure_Months'] = 500.0     # add outlier on purpose for t
    df = pd.concat([df, df.head(3)], ignore_index=True) # add duplicates
    df.to_csv(path, index=False)
```

```
# fill standard feature column blocks
for col in ['Gender', 'Tenure_Months', 'Monthly_Charges', 'Total_Charges'
    if col not in df.columns:
        df[col] = np.random.choice(['Male', 24, 50.0, 1200.0, 'No'], size
print("\n=== Task 2: Dataset Dimensions Shape ===")
print(df.shape)
df.head(3)
```

```
Loading telecom dataset...

=== Task 2: Dataset Dimensions Shape ===
(7043, 21)
```

Out[2]:

	customerID	Gender	SeniorCitizen	Partner	Dependents	Tenure_Months	PhoneService	M
0	7590-VHVEG	Female	0	Yes	No	1	No	
1	5575-GNVDE	Male	0	No	No	34	Yes	
2	3668-QPYBK	Male	0	No	No	2	Yes	

3 rows × 21 columns

In [3]:

```
# task 3: fix missing entries
print("=== Task 3: Handling Missing Rows ===")
print("Null counts before cleaning:\n", df.isnull().sum())
# safe numeric type conversions first to prevent type conversion errors
df['Monthly_Charges'] = pd.to_numeric(df['Monthly_Charges'], errors='coer
df['Total_Charges'] = pd.to_numeric(df['Total_Charges'], errors='coerce')
df['Monthly_Charges'] = df['Monthly_Charges'].fillna(df['Monthly_Charges'
df['Total_Charges'] = df['Total_Charges'].fillna(df['Total_Charges'].medi
print("Null entries remaining:", df.isnull().sum().sum())
# task 4: drop duplicates
print("\n=== Task 4: Removing Duplicate Entries ===")
print("Row count before:", df.shape)
df = df.drop_duplicates()
print("Row count after:", df.shape)
# task 5: clean up text spelling variants
print("\n=== Task 5: Standardizing Text Spelling ===")
df['Gender'] = df['Gender'].astype(str).str.capitalize()
df['Churn_Label'] = df['Churn_Label'].astype(str).str.capitalize()
print("Clean labels in Gender:", df['Gender'].unique())
```

```
=== Task 3: Handling Missing Rows ===
Null counts before cleaning:
 customerID          0
Gender               0
SeniorCitizen        0
Partner              0
Dependents           0
Tenure_Months        0
PhoneService         0
MultipleLines        0
InternetService      0
OnlineSecurity       0
OnlineBackup         0
DeviceProtection     0
TechSupport          0
StreamingTV          0
StreamingMovies      0
Contract             0
PaperlessBilling     0
PaymentMethod        0
Monthly_Charges      0
Total_Charges        0
Churn_Label          0
dtype: int64
Null entries remaining: 0

=== Task 4: Removing Duplicate Entries ===
Row count before: (7043, 21)
Row count after: (7043, 21)

=== Task 5: Standardizing Text Spelling ===
Clean labels in Gender: ['Female' 'Male']
```

In [4]:

```python
# task 6: cast features to proper datatype formats
print("=== Task 6: Casting Feature Data Types ===")
df['Tenure_Months'] = df['Tenure_Months'].astype(int)
print(df[['Tenure_Months', 'Monthly_Charges', 'Total_Charges']].dtypes)
# task 7: filter out outlier records using IQR box plot logic
print("\n=== Task 7: Handling Outliers ===")
print("Max Tenure before outlier filter:", df['Tenure_Months'].max())
q1 = df['Tenure_Months'].quantile(0.25)
q3 = df['Tenure_Months'].quantile(0.75)
iqr = q3 - q1
upper_limit = q3 + 1.5 * iqr
# filter rows keeping values within valid limits
df = df[df['Tenure_Months'] <= upper_limit]
print("Max Tenure after filtering out outliers:", df['Tenure_Months'].max
```

```
=== Task 6: Casting Feature Data Types ===
Tenure_Months        int64
Monthly_Charges    float64
Total_Charges      float64
dtype: object

=== Task 7: Handling Outliers ===
Max Tenure before outlier filter: 72
Max Tenure after filtering out outliers: 72
```

In [5]:
```
# task 8: construct a new tracking column ratio feature
print("=== Task 8: Engineering New Feature ===")
df['Charges_Per_Month'] = df['Total_Charges'] / (df['Tenure_Months'] + 1)
print(df[['Tenure_Months', 'Total_Charges', 'Charges_Per_Month']].head(3)
# task 9: scale numerical data to balance variations (mean=0, std=1)
print("\n=== Task 9: Scaling Continuous Columns ===")
scaler = StandardScaler()
scale_cols = ['Tenure_Months', 'Monthly_Charges', 'Total_Charges', 'Charg
df_scaled = df.copy()
df_scaled[scale_cols] = scaler.fit_transform(df[scale_cols])
print(df_scaled[scale_cols].head(3))
```

```
=== Task 8: Engineering New Feature ===
   Tenure_Months  Total_Charges  Charges_Per_Month
0              1          29.85          14.925000
1             34        1889.50          53.985714
2              2         108.15          36.050000

=== Task 9: Scaling Continuous Columns ===
   Tenure_Months  Monthly_Charges  Total_Charges  Charges_Per_Month
0      -1.277445        -1.160323      -0.994242          -0.757979
1       0.066327        -0.259629      -0.173244          -0.117801
2      -1.236724        -0.362660      -0.959674          -0.411755
```

In [6]:
```
# task 10: split processed data array attributes into 80-20 partition seg
X = df_scaled[scale_cols]
y_mapped = df_scaled['Churn_Label'].map({'Yes': 1, 'No': 0, 'Yes': 1, 'No
y = y_mapped.fillna(0).astype(int)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
print("=== Task 10: Final Matrix Target Shapes ===")
print("Training Inputs (X_train) Shape:", X_train.shape)
print("Testing Inputs (X_test) Shape:", X_test.shape)
# task 11: export full processed data frame to a fresh local file
export_path = r"C:\Users\muthi\OneDrive\Desktop\PRACTICAL EXAM\ML\Clean_T
df_scaled.to_csv(export_path, index=False)
print("\n=== Task 11: Export File Verification ===")
print(f"Dataset successfully compiled and saved at:\n{export_path}")
```

```
=== Task 10: Final Matrix Target Shapes ===
Training Inputs (X_train) Shape: (5634, 4)
Testing Inputs (X_test) Shape: (1409, 4)

=== Task 11: Export File Verification ===
Dataset successfully compiled and saved at:
C:\Users\muthi\OneDrive\Desktop\PRACTICAL EXAM\ML\Clean_Telecom_Churn.csv
```